
Lista de Exercícios 2

Criação de programas em C++ utilizando os conceitos abordados na Aula 03.

Arrays unidimensionais

1. Escreva um programa que declare um array para armazenar 10 números inteiros. O programa deve solicitar a entrada dos valores para o usuário e armazenar dentro desse array. Em seguida, apresente na tela os valores armazenados dentro do array.
2. Escreva um programa que receba 10 valores reais e armazene dentro de um vetor. Em seguida, apresente na tela os valores armazenados dentro do array, exibindo-os em ordem inversa.
3. Escreva um programa que receba 15 valores reais, correspondentes às notas que um aluno obteve na realização de suas atividades. Em seguida, apresente na tela todas essas notas e o valor da média que o aluno conseguiu tirar. Considere a exibição dos valores utilizando duas casas decimais.
4. Escreva um programa que solicite para o usuário fornecer um array de 10 caracteres. O programa deve apresentar o array fornecido pelo usuário, juntamente com o total de vogais e o total de consoantes que existem dentro desse array.
5. Escreva um programa que solicite para o usuário fornecer 20 números inteiros e armazene esses valores dentro de um array. O programa deve separar os números pares e armazená-los dentro de um array denominado <pares>. Repita o processo para os números ímpares e armazene-os dentro de um array denominado <ímpares>. Apresente na tela os valores que estão armazenados dentro de cada array.

6. Escreva um programa que solicite a entrada de quatro notas de 10 alunos. O programa deve calcular e armazenar a média de cada aluno dentro de um outro array. Apresente o resultado, informando a quantidade de alunos que conseguiram média maior ou igual a 7. Considere a exibição dos valores utilizando duas casas decimais.

7. Escreva um programa que solicite a entrada de 5 valores inteiros e armazene-os dentro de um array. Em seguida, apresente o resultado da soma e da multiplicação desses valores.

8. Escreva um programa que solicite a entrada da idade e da altura de 10 pessoas. Armazene as informações dentro de vetores específicos. Em seguida, apresente os valores da idade e da altura de cada pessoa, na ordem inversa da que foram fornecidas.

9. Escreva um programa que solicite a entrada dos valores para um vetor de 10 números inteiros. Em seguida, apresente os valores que foram fornecidos, juntamente com o cálculo do quadrado de cada elemento. Após apresentar essas informações, exiba o resultado da soma dos elementos do vetor assim como da soma dos quadrados desses elementos.

10. Escreva um programa que solicite a entrada dos valores para dois vetores de 10 números inteiros cada. Em seguida, gere um terceiro vetor, o qual deve ser preenchido com os valores intercalados dos dois vetores que foram fornecidos pelo usuário. Apresente os valores dos vetores fornecidos pelo usuário, um ao lado do outro. Em seguida, apresente os valores do terceiro vetor, um na frente do outro.

11. Um professor de Educação Física anotou as idades e alturas de 40 alunos de todas as suas quatro turmas. Escreva um programa que armazene esses valores dentro de vetores específicos para cada uma dessas turmas. Em seguida, para cada uma delas, determine a quantidade de alunos maiores de 13 anos de idade que possuem altura inferior à média de altura da sua turma. Apresente um resultado, informando os seguintes valores: média de altura dos alunos de cada turma, total de alunos maiores de 13 anos de idade que cada turma possui e média de altura desses alunos dentro de sua própria turma. Considere a exibição dos valores referentes à altura utilizando três casas decimais.

12. Escreva um programa que receba um número indeterminado de valores inteiros, maiores ou iguais ao valor zero. Considere que o limite de valores que podem ser armazenados no vetor seja 100 e que a entrada de valores deve ser interrompida quando o usuário fornecer um valor negativo. Em seguida, o programa deve apresentar um menu de opções que permita ao usuário realizar as seguintes ações:

1. Apresentar a quantidade de valores que foram fornecidos;
2. Apresentar todos os valores na ordem em que foram informados, um na frente do outro;
3. Apresentar todos os valores em ordem inversa, um na frente do outro;

4. Calcular e apresentar a soma dos valores;
5. Calcular e apresentar a média dos valores;
6. Calcular e apresentar a quantidade de valores que estejam acima da média;
7. Calcular e apresentar a quantidade de valores que sejam menores ou iguais ao valor cinco;
8. Encerrar o programa apresentando uma mensagem personalizada.

13. Escreva um programa que receba a temperatura média de cada mês e armazene em um array de números reais. Apresente todas as temperaturas que foram fornecidas. Em seguida, o programa deve calcular a média anual das temperaturas e exibir todas aquelas que estiverem acima da média anual. Apresente o valor da temperatura e o nome do mês em que ela ocorreu, conforme o exemplo abaixo. Observe que o nome do mês deve ser exibido por extenso (Janeiro, Fevereiro, etc.). Considere a exibição dos valores utilizando duas casas decimais.

```
1  Temperaturas mensais: [30.50 25.00 24.10 24.00 22.50 21.00 18.50 25.00 26.50 27.80 29.50 27.20]
2
3  Média anual: 25.13
4
5  * Meses com temperatura acima da média:
6
7  Janeiro Setembro Outubro Novembro Dezembro
8    30.50      26.50      27.80      29.50      27.20
```

14. Walter Paraíba costuma realizar o pagamento dos vendedores de sua lojinha de R\$ 1,99 com base em comissões. Cada vendedor recebe R\$ 1000,00 por semana, mais 5% do valor de suas vendas brutas durante aquela semana. Por exemplo, um vendedor que teve vendas brutas no valor de R\$ 3000,00 recebe mais 5% desse valor, ou seja, um total de R\$ 1.150,00. Para controlar a quantidade de vendedores que devem receber uma comissão, Walter Paraíba utiliza uma planilha do Excel. Porém, visando automatizar a sua empresa, ele deseja que você escreva um programa em C++ para determinar o total de vendedores que realizaram vendas brutas onde o valor do salário final esteve dentro dos seguintes intervalos:

1. Abaixo de R\$ 1.150,00:
2. Entre R\$ 1.150,00 - R\$ 1.199,00
3. Entre R\$ 1.200,00 - R\$ 1.299,00
4. Entre R\$ 1.300,00 - R\$ 1.399,00
5. Entre R\$ 1.400,00 - R\$ 1.499,00
6. Acima de R\$ 1.500,00

A lojinha de Walter Paraíba encontra-se em fase de expansão e atualmente possui 10 vendedores. Escreva um programa que solicite o valor da venda bruta realizada por cada vendedor durante a semana. O programa deve apresentar como resultado o valor do salário que deverá ser pago aos vendedores, de acordo com cada intervalo listado anteriormente, juntamente com o total

de vendedores que deverão receber esse salário. Para demonstrar que você é um excelente programador, utilize uma fórmula para determinar o valor da comissão, sem recorrer a diversos *if-else* aninhados. Considere a exibição dos valores utilizando duas casas decimais. Observe um exemplo de saída do programa:

```
1 * Lojinha de R$ 1,99 do Walter Paraíba
2
3 => Informe o valor da venda bruta de cada vendedor:
4
5 1: 2000.00
6 2: 3000.00
7 3: 4000.00
8 4: 2700.00
9 5: 3750.00
10 6: 4350.00
11 7: 5550.00
12 8: 500.00
13 9: 8000.00
14 10: 4100.00
15
16 => Total de vendedores dentro de cada faixa salarial:
17
18 - Abaixo de R$ 1.150,00: 4
19 - Entre R$ 1.150,00 e R$ 1.199,00: 2
20 - Entre R$ 1.200,00 e R$ 1.299,00: 3
21 - Entre R$ 1.300,00 e R$ 1.399,00: 0
22 - Entre R$ 1.400,00 e R$ 1.499,00: 1
23 - Acima de R$ 1.500,00: 0
```

15. Em uma competição de salto em distância, cada atleta tem direito a realizar cinco saltos consecutivos. O resultado final será determinado pela média desses valores. Escreva um programa que receba o nome do atleta e as cinco distâncias alcançadas por ele durante cada um de seus saltos. Em seguida, o programa deve apresentar esses valores juntamente com o valor da média obtida para os saltos que o atleta realizou. O programa deve ser encerrado quando não for informado o nome do atleta. O resultado do programa deve ser apresentado conforme o exemplo abaixo:

```
1  Atleta: Paulo Giovanni
2
3  Primeiro Salto: 7.0
4  Segundo Salto: 7.4
5  Terceiro Salto: 10.2
6  Quarto Salto: 9.4
7  Quinto Salto: 18.5
8
9  => Resultado final:
10
11 Atleta: Paulo Giovanni
12 Saltos: 7.0 m, 7.4 m, 10.2 m, 9.4 m e 18.5 m
13 Média dos saltos: 10.5 m
```

16. Walter Paraíba resolveu novamente expandir seus negócios e adquiriu a rede de televisão do apresentador Sílvio Santos, de quem ele é um grande fã. Com o objetivo de dar continuidade a automatização de suas empresas e melhorar a qualidade dos serviços oferecidos, ele quer realizar uma enquete entre seus telespectadores, para saber qual o melhor jogador de futebol após cada partida que for transmitida pela sua nova emissora de televisão, denominada de SWPC, ou *Sistema Walter Paraíba de Telecomunicações*. Para que a divisão de esportes de sua emissora possa computar os votos dos telespectadores sem utilizar uma planilha do Excel, você foi contratado para desenvolver um programa utilizando a linguagem C++. Para votar, cada telespectador deve realizar uma chamada telefônica e informar o número da camiseta do jogador que ele considera que foi o melhor em campo. Considere que nos jogos transmitidos pela SWPC, os jogadores utilizam camisetas com números entre 1 e 25. Se o telespectador informar um número inválido, o programa deve ignorá-lo e exibir uma breve mensagem de aviso. Em seguida, deve continuar a receber os votos dos telespectadores. Caso seja informado o número zero, a votação estará encerrada. Após o fim da votação, o programa deverá apresentar um resultado, contendo as seguintes informações:

1. O total de votos computados;
2. Os números e respectivos votos de todos os jogadores que receberam votos;
3. O percentual de votos de cada um desses jogadores;
4. O número do jogador escolhido como o melhor jogador da partida, juntamente com o número de votos que ele obteve e o percentual desses votos.

Observe que os votos inválidos e o zero final não devem ser computados como votos. O resultado final deve ser apresentado ordenado pelo número do jogador. O programa deve utilizar arrays. Para calcular o percentual de cada jogador, o programa deve utilizar uma função. Essa função receberá dois parâmetros: o número de votos de um jogador e o total de votos. Ela deve calcular o percentual de votos do jogador e retornar esse valor. A saída apresentada pelo

programa deve seguir o exemplo abaixo:

```
1
2 => Enquete Sistema Walter Paraíba de Telecomunicações
3
4 * Quem foi o melhor jogador da partida?
5
6 - Tecle o número do jogador <0 para encerrar>: 9
7 - Tecle o número do jogador <0 para encerrar>: 10
8 - Tecle o número do jogador <0 para encerrar>: 9
9 - Tecle o número do jogador <0 para encerrar>: 10
10 - Tecle o número do jogador <0 para encerrar>: 11
11 - Tecle o número do jogador <0 para encerrar>: 10
12 - Tecle o número do jogador <0 para encerrar>: 50
13
14 - Atenção: informe somente valores entre 1 e 25, ou
15 tecle 0 para encerrar a votação!
16
17 - Tecle o número do jogador <0 para encerrar>: 9
18 - Tecle o número do jogador <0 para encerrar>: 9
19 - Tecle o número do jogador <0 para encerrar>: 0
20
21 => Resultado da votação:
22
23 - Foram computados 8 votos.
24
25 Jogador Votos Percentual
26      9      4      50.0%
27     10      3      37.5%
28     11      1      12.5%
29
30 - O melhor jogador da partida foi o número 9! Ele
31 obteve um total de 4 votos, o que corresponde a 50%
32 do total de votos válidos.
```

17. Para informatizar suas empresas, Walter Paraíba decidiu realizar uma enquete para verificar

qual o melhor sistema operacional para ser utilizado em seus servidores. Até o momento, ele pensava que seria o Microsoft Excel... Para isso, ele contratou você para desenvolver um programa em linguagem de programação C++, que colete os dados e faça a tabulação dos resultados. Ao ser executado, o programa deve apresentar um menu inicial, conforme o seguinte exemplo:

```
1 * Enquete: Qual o melhor Sistema Operacional que pode
2 ser utilizado nos servidores das empresas de Walter
3 Paraíba?
4
5 1. Windows Server
6 2. Unix
7 3. Linux
8 4. Netware
9 5. Mac OS
10 6. Microsoft Excel
11 7. Outro
12 0. Encerrar votação
13
14 => Informe o seu voto: 6
```

Você deve para desenvolver um programa que leia a entrada dos usuários e informe o resultado final da enquete. Somente deverá ser aceito valores válidos. Os valores referentes a cada uma das opções devem ser armazenados em um array de números inteiros. Após os dados terem sido completamente informados, o programa deverá calcular a percentual de cada um dos concorrentes e informar o vencedor da enquete. O resultado do programa deverá ser apresentado conforme o exemplo abaixo:

```
1 -----
2 Sistema Operacional      Votos      %
3 -----
4 Windows Server          1150      12.55%
5 Unix                    3800      41.48%
6 Linux                   3750      40.93%
7 Netware                  300       3.27%
8 Mac OS                   110       1.20%
9 Microsoft Excel           1       0.01%
```

```

10 Outro                                50      0.55%
11 -----
12 Total                                9161     100%
13 -----
14
15 => O Sistema Operacional mais votado durante a
16 enquete foi o Unix. Ele obteve um total de 3800
17 votos, o que corresponde a 41.48% dos votos válidos.

```

18. Escrever um programa para ler 10 valores inteiros em um vetor A. Multiplicar por 3 os valores inseridos no vetor A e armazená-los em um vetor B. Exibir na tela os valores de ambos os vetores, uma linha de cada vez, separando os valores por um espaço.

19. Escrever um programa para ler 10 valores inteiros em um vetor A. Escrever uma função para calcular o fatorial dos valores inseridos no vetor A e armazená-los em um vetor B. Exibir na tela os valores de ambos os vetores, uma linha de cada vez, separando os valores por um espaço.

20. Como reconhecimento ao bom resultado alcançado na venda dos produtos de suas lojinhas, Walter Paraíba resolveu dar um abono para seus funcionários. Ao invés de realizar os cálculos no Excel, ele contratou você para desenvolver uma aplicação em linguagem C++, que servirá como uma ferramenta para projetar quanto será o gasto que ele terá que desembolsar com o pagamento desses valores. Após diversas reuniões realizadas junto com a diretoria executiva, a diretoria financeira e os representantes do sindicato, Walter Paraíba chegou a seguinte conclusão:

- a. Cada funcionário receberá o equivalente a 20% do seu salário bruto de dezembro;
- b. O piso do abono será de R\$ 100 reais, isto é, aqueles funcionários cujo salário for muito baixo, recebem este valor mínimo.

Neste momento, não se deve ter nenhuma preocupação com colaboradores com tempo menor de casa, descontos, impostos ou outras particularidades. A aplicação deverá permitir a entrada dos valores de diversos salários, até que um valor igual a zero encerre a entrada de dados. Em seguida, o programa deverá calcular o valor do abono concedido a cada colaborador, de acordo com a regra definida acima. Após os cálculos, deve ser apresentado as seguintes informações:

1. O salário de cada funcionário, juntamente com o valor do abono que deve ser concedido;
2. O número total de funcionários processados;
3. O valor total a ser gasto com o pagamento do abono;
4. O número de funcionários que receberá o valor mínimo de 100 reais;
5. O maior valor pago como abono.

Considere que os valores devem ser apresentados com duas casas decimais. Para facilitar o entendimento do que deve ser desenvolvido, Walter Paraíba disponibilizou um exemplo ilustrativo para a saída que deve ser exibida pelo programa:

```
1  =====
2  Projeção de Gastos com Abono Salarial
3  =====
4
5  => Por gentileza, informe os valores salariais
6  (digite 0 para finalizar):
7
8  1. Salário: 1000
9  2. Salário: 300
10 3. Salário: 500
11 4. Salário: 100
12 5. Salário: 4500
13 6. Salário: 0
14
15 => Resultado dos abonos que devem ser concedidos:
16
17      Salário          Abono
18 R$ 1000.00 - R$ 200.00
19 R$ 300.00 - R$ 100.00
20 R$ 500.00 - R$ 100.00
21 R$ 100.00 - R$ 100.00
22 R$ 4500.00 - R$ 900.00
23
24 => Resumo das informações:
25
26 * Foram processados 5 colaboradores
27 * Total gasto com abonos: R$ 1400.00
28 * Valor mínimo pago a 3 colaboradores
29 * Maior valor de abono pago: R$ 900.00
```

21. Walter Paraíba deseja contratar um estagiário para trabalhar no Suporte de Informática. A primeira tarefa que esse estagiário deverá realizar será testar todos os teclados que existem

no departamento. O estagiário deverá anotar o estado de cada um deles, para que seja possível verificar a necessidade de se realizar alguma troca. Considere que os teclados podem estar em perfeito estado, quebrados, sujos ou com problemas no cabo. Desenvolva um programa que receba um número indeterminado de entradas e indique o tipo de defeito, juntamente com o percentual em relação a todos os teclados que foram testados. Apresente o resultado conforme o modelo abaixo:

```

1
2 => Quantidade de teclados: 100
3
4 -----
5             Situação      Quantidade Percentual
6 -----
7 1. Perfeito estado          40          40%
8 2. Sujos                    30          30%
9 3. Quebrados                15          15%
10 4. Problemas no cabo        15          15%
11 -----

```

22. Escreva um programa que solicite a entrada de cinco modelos de carros e o consumo de combustível de cada um deles, ou seja, quantos quilômetros cada um desses carros faz com um litro de combustível. Em seguida, o programa deve solicitar a entrada do preço do litro da gasolina. Após isso, o programa deve calcular e exibir as seguintes informações:

1. O modelo do carro mais econômico;
2. Quantos litros de combustível cada veículo consome para percorrer uma distância de 1000 quilômetros e quanto isto vai custar.

Abaixo segue uma tela de exemplo. O disposição das informações deve ser o mais próxima possível ao exemplo. Os dados são fictícios e podem mudar a cada execução do programa. Considere a apresentação dos valores utilizando duas casas decimais.

```

1
2 -----
3 Comparativo de Consumo de Combustível
4 -----
5
6 >> Informe os dados dos veículos e do total de
7 quilômetros que ele realiza com um litro de gasolina:

```

```

8
9 * Veículo 1: Fusca
10 * Km por litro: 7
11
12 * Veículo 2: Gol
13 * Km por litro: 10
14
15 * Veículo 1: Uno
16 * Km por litro: 12.5
17
18 * Veículo 1: Vectra
19 * Km por litro: 9
20
21 * Veículo 1: Peugeot
22 * Km por litro: 714.5
23
24 >> Relatório Final
25
26 +-----+-----+-----+-----+
27 | Automóvel | Consumo | 1000 km | Total R$ |
28 +-----+-----+-----+-----+
29 | 1. Fusca   | 7.0 | 142.9 litros | 321.43 |
30 | 2. Gol     | 10.0 | 100.0 litros | 225.00 |
31 | 3. Uno     | 12.5 | 80.0 litros  | 180.00 |
32 | 4. Vectra  | 9.0 | 111.1 litros | 250.00 |
33 | 5. Peugeot | 14.5 | 69.0 litros  | 155.17 |
34 +-----+-----+-----+-----+
35
36 >> Modelo de carro mais econômico: Peugeot

```

23. Escreva um programa que simula a rolagem de dois dados. O programa deve utilizar a função *rand* para rolar o primeiro dado e deve utilizá-la novamente, para rolar o segundo dado. A soma dos dois valores deve então ser calculada. Atenção: cada dado pode mostrar um valor inteiro de 1 a 6, portanto a soma dos dois valores deverá variar de 2 a 12, com 7 sendo a soma mais frequente, e 2 e 12 sendo as somas menos frequentes. A Figura 1 ilustra as 36 possíveis combinações de dois dados. Seu programa deve rolar os dois dados 36.000 vezes. Utilize um

array unidimensional para contar o número de vezes que cada possível soma aparece. Imprima os resultados em um formato tabular. Além disso, o programa também deverá verificar se os totais são razoáveis, ou seja, como existem seis maneiras de rolar um 7, então aproximadamente um sexto de todas as rolagens deve ser 7.

	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	10
5	6	7	8	9	10	11
6	7	8	9	10	11	12

Figura 1: 36 possíveis resultados, obtidos após a rolagem de dois dados.

24. O empresário em ascensão, Walter Paraíba, adquiriu uma pequena companhia aérea a qual denominou de *Aerolinhas Paraíba*. Visando melhorar o atendimento de sua nova empresa, ele contratou você para automatizar seu sistema de reservas. Você precisa desenvolver um programa para atribuir assentos em cada voo da companhia aérea. A capacidade máxima é de 10 assentos, pois a empresa ainda é bem pequena. O programa deve exibir um menu de alternativas, onde o cliente poderá escolher o tipo de passagem que deseja comprar: “Primeira Classe” ou “Econômica”. O menu também deve possuir uma opção para que o cliente possa visualizar os assentos disponíveis. Se o cliente digitar 1, ele poderá comprar um assento na primeira classe (numerados de 1 até 5). Se o cliente digitar 2, ele poderá comprar um assento na classe econômica (numerados de 6 até 10). O programa deve imprimir um bilhete de embarque, indicando a classe escolhida, o nome do cliente e o número do assento que ele escolheu, conforme o modelo apresentado abaixo. Utilize um array unidimensional para representar o gráfico de assentos do avião. Inicialize todos os elementos do array como 0, para indicar que todos os assentos estão vazios. À medida que cada assento é atribuído, configure os elementos correspondentes do array como “x”, para indicar que o assento não está mais disponível. Naturalmente, o programa nunca deverá permitir a compra um assento que já foi atribuído para algum cliente. Quando a primeira classe estiver lotada, o programa deve perguntar ao cliente se ele aceita ficar na classe econômica, e vice-versa. Se ele aceitar, faça a atribuição apropriada de assento. Caso contrário, imprima uma mensagem informando que o próximo voo parte em 3 horas.

```
1  -----
2  >> Aerolinhas Paraíba <<
3  -----
4
5  # Menu Principal
6
7  1. Primeira classe
8  2. Segunda classe
9  3. Exibir assentos disponíveis
10
11 >> Por favor, informe a opção desejada: 3
12
13 => Assentos disponíveis até o momento:
14
15 * Primeira Classe:
16
17 [1] [2] [3] [4] [5]
18 [0] [x] [x] [0] [0]
19
20 * Segunda Classe:
21
22 [6] [7] [8] [9] [10]
23 [x] [x] [0] [0] [0]
24
25 # Menu Principal
26
27 1. Primeira classe
28 2. Segunda classe
29 3. Exibir assentos disponíveis
30
31 >> Por favor, informe a opção desejada: 1
32
33 => Digite seu nome: Walter Paraíba
34 => Digite o número do assento: 2
35
```

```
36 >> Atenção: esse assento não está mais disponível!
37
38 # Menu Principal
39
40 1. Primeira classe
41 2. Segunda classe
42 3. Exibir assentos disponíveis
43
44 >> Por favor, informe a opção desejada: 1
45
46 => Digite seu nome: Walter Paraíba
47 => Digite o número do assento: 1
48
49 >> Compra realizada com sucesso! Confira as
50 informações do seu bilhete de embarque:
51
52 +-----+
53 | Aerolinhas Paraíba: Primeira Classe |
54 +-----+
55 | * Passageiro: Walter Paraíba      |
56 | * N. Assento: 1                   |
57 |                                   |
58 | >> Tenha uma boa viagem!          |
59 +-----+
```

25. Escreva um programa que realize a leitura de um array de números reais. O programa deve solicitar o total de elementos do array. Em seguida, ele deve exibir uma mensagem na tela, informando os números que compõem a sequência e o total de vezes que cada um deles aparece dentro do array, conforme o exemplo abaixo:

```
1 * Informe o total de elementos: 5
2 * Informe os valores: 3.0 3.0 3.0 1.5 0.0
3
4 - Valor 3.0 -> 3 vezes
5 - Valor 1.5 -> 1 vez
6 - Valor 0.0 -> 1 vez
```

Arrays bidimensionais

26. Escreva um programa que receba a entrada de uma matriz de números inteiros, com tamanho 4 x 4. Em seguida, o programa deve contar e exibir quantos valores maiores que 10 ela possui.
27. Declare uma matriz 5 x 5. Preencha sua diagonal principal com o valor 1. Para as demais posições, preencha com o valor 0. Em seguida, apresente a matriz na tela.
28. Escreva um programa que preencha uma matriz 4 x 4 com o produto do valor da linha e da coluna de cada elemento. Em seguida, imprima na tela a matriz.
29. Escreva um programa que receba os valores de uma matriz 4 x 4. Em seguida, imprima a matriz e retorne a localização (linha e coluna) do maior valor.
30. Escreva um programa que receba a entrada dos valores de uma matriz 5 x 5. Em seguida, solicite a entrada de um valor, do mesmo tipo de dados da matriz. O programa deve realizar uma busca desse valor na matriz e, ao final, escrever a sua localização (linha e coluna). Caso contrário, o programa deve exibir uma mensagem informando que valor não está armazenado na matriz.
31. Escreva um programa que receba a entrada de duas matrizes 4 x 4 e escreva uma terceira matriz com os maiores valores de cada posição das matrizes que foram lidas. Apresente os valores de todas as matrizes.
32. Escreva um programa que gere e apresente uma matriz de tamanho 10 x 10. Os valores de seus elementos devem ser preenchidos utilizando as seguintes definições:
- $$A[i][j] = 2i + 7j - 2, \text{ se } i < j;$$
- $$A[i][j] = 3i^2 - 1, \text{ se } i = j;$$
- $$A[i][j] = 4i^3 - 5j^2 + 1, \text{ se } i > j.$$
33. Escreva um programa que receba uma matriz de 3 x 3 elementos. Calcule e apresente a soma dos elementos que estão acima da diagonal principal.
34. Escreva um programa que receba uma matriz de 3 x 3 elementos. Calcule e apresente a soma dos elementos que estão abaixo da diagonal principal.
35. Escreva um programa que receba uma matriz de 3 x 3 elementos. Calcule e apresente a soma dos elementos que estão na diagonal principal.

36. Escreva um programa que receba uma matriz de 3 x 3 elementos. Calcule e imprima a sua transposta.
37. Escreva um programa que gere uma matriz 4 x 4 com valores no intervalo [1, 20]. Escreva um programa que transforme a matriz gerada numa matriz triangular inferior, ou seja, atribuindo zero a todos os elementos acima da diagonal principal. Imprima a matriz original e a matriz transformada.
38. Escreva um programa que gere automaticamente números entre 0 e 99, para uma cartela de BINGO. Cada cartela deve conter 5 linhas de 5 números. O programa deve gerar os dados, de maneira que os números não sejam repetidos dentro da cartela. Em seguida, exiba a cartela do BINGO na tela.
39. Escreva um programa que receba os valores de uma matriz 5 x 10, referente às respostas de 5 candidatos para 10 questões de múltipla escolha. Em seguida, o programa deve receber um vetor de 10 posições, correspondente ao gabarito de respostas. Considere que as respostas podem ter os seguintes valores: *a*, *b*, *c* ou *d*. Em seguida, o programa deve comparar as respostas dos candidatos de acordo com os valores do gabarito. O programa deve apresentar um resultado, informando o total de questões que cada candidato acertou.
40. Escreva um programa que receba uma matriz 10 x 3, contendo as notas que 10 alunos tiveram em 3 provas. O programa deve apresentar um resultado, exibindo o número de alunos que tiveram a pior nota em cada uma das provas.
41. Escreva um programa que receba os valores de uma matriz de números inteiros, de tamanho 3 x 3. Em seguida, o programa deve gerar e apresentar na tela um array unidimensional, contendo a soma dos valores de cada coluna da matriz. Por exemplo, considerando a matriz abaixo:

1	5	-8	10
2	1	2	15
3	25	10	7

O programa devera exibir um array, contendo os valores 31, 4 e 32.

42. Escreva um programa que receba uma matriz de números inteiros, com 5 linhas e 4 colunas, contendo as seguintes informações:

1. Primeira coluna: número de matrícula;
2. Segunda coluna: média das provas;
3. Terceira coluna: média dos trabalhos;

4. Quarta coluna: nota final.

O programa deve realizar as seguintes ações:

- a. Realizar a leitura das três primeiras informações de cada aluno;
- b. Calcular a nota final, como sendo a soma da média das provas e da média dos trabalhos;
- c. Exibir a matrícula dos alunos que tiveram a maior nota final;
- d. Exibir a média aritmética das notas finais.

43. Escreva um programa que receba uma matriz de valores reais, de tamanho 3 x 6. Em seguida, o programa deve realizar as seguintes ações:

- a. Imprimir a soma de todos os elementos das colunas ímpares;
- b. Imprimir a média aritmética dos elementos da segunda e da quarta coluna;
- c. Substituir os valores da sexta coluna pela soma dos valores das colunas 1 e 2;
- d. Imprimir os valores da matriz, após essas alterações.

44. Escreva um programa que receba duas matrizes de números reais, de tamanho 2 x 2. Em seguida, o programa deve apresentar um menu com as seguintes opções:

1. Somar os valores das duas matrizes;
2. Subtrair os valores da primeira matriz pelos valores da segunda matriz;
3. Ler um valor e adicioná-los aos elementos de ambas as matrizes;
4. Imprimir os valores de cada matriz.

Nas duas primeiras opções, uma terceira matriz deverá ser criada. Na terceira opção, o valor da constante deverá ser adicionado aos valores de cada matriz que foi lida anteriormente.

45. Escreva um programa que faça a leitura dos valores de duas matrizes A e B, de tamanho 3 x 3. O programa deve multiplicar as duas matrizes e armazenar o resultado na matriz C. Em seguida, imprima os valores de cada uma das matrizes.

46. Escreva um programa que faça a leitura dos valores da matriz A, de tamanho 3 x 3. O programa gerar a matriz B, utilizando a fórmula $B = A^2$. Em seguida, imprima os valores de cada uma das matrizes.

47. Escreva um programa que receba a coordenada X e Y de um ponto e um número N de pontos ($1 < N < 100$). O programa deve solicitar a entrada da coordenada X e Y de para esses N pontos, utilizando-se valores reais. Em seguida, o programa deve calcular e imprimir, sem repetição, os raios das circunferências centradas no ponto (x, y) , que passam por pelo menos um dos N pontos que foram fornecidos. Observações:

- a. A distância entre os pontos (a, b) e (c, d) é dada por $\sqrt{(a - c)^2 + (b - d)^2}$;
- b. Dois pontos estão na mesma circunferência se estão à mesma distância do centro.

Para facilitar, considere o seguinte o seguinte exemplo de execução do programa:

```

1 >> Informe a coordenada (X,Y) do ponto: 1.0 1.0
2 >> Informe o total de pontos: 5
3 >> Informe as coordenadas (X, Y) dos 5 pontos:
4
5 [1]: -1.0 1.2
6 [2]: 1.5 2.0
7 [3]: 0.0 -2.0
8 [4]: 0.0 0.5
9 [5]: 4.0 2.0
10
11 * Raios das circunferências que passam pelo ponto
12 (1.0, 1,0): 1.12, 2.01 e 3.162

```

48. Escreva um programa que faça a leitura de uma matriz de números inteiros, com L linhas e C colunas. Os valores de L e C devem ser solicitados ao usuário. Em seguida, o programa deve calcular a soma da sequência de números inteiros armazenada em cada coluna. Exibir a matriz e o resultado dessa soma, conforme o exemplo abaixo:

```

1 >> Informe o total de linhas da matriz: 2
2 >> Informe o total de colunas da matriz: 8
3
4      8      2      4      3      4      2      5      1
5 +    3      3      7      5      2      3      3      7
6 -----
7 1    1      6      1      8      6      5      8      8

```

49. Uma empresa tem quatro equipes de vendas (1 a 4) que vendem cinco produtos diferentes (1 a 5). Uma vez por dia, cada vendedor passa uma nota de cada tipo de produto diferente vendido. Cada nota contém o seguinte:

- a. O número do vendedor;
- b. O número do produto;
- c. O valor total em reais desse produto, vendido nesse dia.

Portanto, cada vendedor passa entre 0 e 5 notas de vendas por dia. Suponha que as informações a partir de todas as notas durante o último mês estão disponíveis. Escreva um programa que leia todas essas informações para as vendas do último mês e apresente um resumo com as vendas totais por vendedor e produto. Todos os totais devem ser armazenados em uma matriz bidimensional denominada de *sales*. Depois de processar todas as informações durante o último mês, imprima os resultados em formato tabular, com cada uma das colunas representando um vendedor específico e cada uma das linhas representando um produto específico. Cruze cada linha de total para obter as vendas totais de cada produto durante o último mês. Cruze cada coluna de total para obter as vendas totais por vendedor durante o último mês. A saída apresentada deve incluir esses totais cruzados à direita das linhas totalizadas e na parte inferior das colunas totalizadas.

50. Escreva um programa que faça a leitura de uma matriz de números inteiros, com L linhas e C colunas. Os valores de L e C devem ser solicitados ao usuário. Em seguida, o programa deve apresentar a matriz e exibir uma mensagem informando o número de linhas e de colunas nulas que ela possui. Para facilitar, observe o exemplo abaixo:

```
1 >> Informe o total de linhas da matriz: 4
2 >> Informe o total de colunas da matriz: 4
3 >> Informe os valores da matriz:
4
5 1 0 2 3
6 ...
7
8 * Valores da matriz:
9
10 +---+---+---+---+
11 | 1 | 0 | 2 | 3 |
12 +---+---+---+---+
13 | 4 | 0 | 5 | 6 |
14 +---+---+---+---+
15 | 0 | 0 | 0 | 0 |
16 +---+---+---+---+
17 | 0 | 0 | 0 | 0 |
18 +---+---+---+---+
19
20 * A matriz possui 2 linhas e 1 coluna com todos os
21 valores nulos!
```

Ordenação e Pesquisa

51. Escreva um programa que receba 10 valores inteiros e armazene dentro de um vetor. Em seguida, o programa deve apresentar uma mensagem informando se o vetor está ou não ordenado de maneira crescente.
52. Escreva um programa que receba a entrada dos valores de um vetor de números inteiros. Considere que o vetor possa armazenar até 20 números. O programa deve ordenar os valores de maneira crescente. Em seguida, o programa deve apresentar os valores do vetor original e também do vetor ordenado.
53. Escreva um programa que receba a entrada dos valores de um vetor de números inteiros. Considere que o vetor possa armazenar até 20 números. O programa deve ordenar os valores de maneira decrescente. Em seguida, o programa deve apresentar os valores do vetor original e também do vetor ordenado.
54. Escreva um programa que receba a entrada dos valores de um vetor de números inteiros. Considere que o vetor possa armazenar até 20 números. Após a entrada dos valores, o programa deve perguntar ao usuário se eles devem ser ordenados de maneira crescente ou decrescente. Realizar a ordenação dos valores e apresentar os valores do vetor original e também do vetor ordenado. Em seguida, o programa deve solicitar a entrada de um número inteiro. O programa deve inserir esse elemento em sua posição correta, dentro do vetor ordenado. Por último, apresentar os valores do vetor ordenado, após a inserção desse elemento.
55. Escreva um programa que receba a entrada dos valores de um vetor de números inteiros. Considere que o vetor possa armazenar até 20 números. O programa deve ordenar os valores de maneira crescente. Em seguida, o programa deve receber um número inteiro e verificar se ele está armazenado dentro do vetor. Se ele estiver armazenado, apresentar uma mensagem informando a sua posição. Caso contrário, apresentar uma mensagem informando que ele não foi localizado.
56. Escreva um programa que receba a entrada dos valores de um vetor de números inteiros. Considere que o vetor possa armazenar até 20 números. O programa deve ordenar os valores de maneira crescente, utilizando o *método de ordenação bolha*. Em seguida, o programa deve apresentar os valores do vetor original e também do vetor ordenado.
57. Escreva um programa que receba a entrada dos valores de um vetor de números inteiros. Considere que o vetor possa armazenar até 20 números. O programa deve ordenar os valores de maneira crescente, utilizando o *método de ordenação por seleção*. Em seguida, o programa deve apresentar os valores do vetor original e também do vetor ordenado.

58. Escreva um programa que receba a entrada dos valores de um vetor de números inteiros. Considere que o vetor possa armazenar até 20 números. O programa deve ordenar os valores de maneira crescente, utilizando o *método Quick Sort*. Em seguida, o programa deve apresentar os valores do vetor original e também do vetor ordenado.
59. Escreva um programa que receba a entrada de 20 nomes e armazene-os dentro de um vetor. O programa deve ordenar esses nomes por ordem alfabética. Em seguida, o programa deve apresentar os valores do vetor original e também do vetor ordenado.
60. Escreva um programa que receba uma *string*. Utilizando o *método de ordenação bolha*, o programa deve colar suas letras em ordem crescente. Em seguida, o programa deve apresentar os valores da string original e também da string ordenada.
61. Escreva um programa que receba a entrada de 20 nomes e armazene-os dentro de um vetor. O programa deve ordenar esses nomes de acordo com o tamanho de cada um, utilizando o *método de ordenação por seleção*. Em seguida, o programa deve apresentar os valores do vetor original e também do vetor ordenado.
62. Escreva um programa que receba uma *string*. Utilizando o *método Quick Sort*, o programa deve colar suas letras em ordem decrescente. Em seguida, o programa deve apresentar os valores da string original e também da string ordenada.
63. Escreva um programa que receba a entrada de 20 nomes e armazene-os dentro de um vetor. Em seguida, o programa deve solicitar a entrada de um nome e verificar se ele está armazenado dentro do vetor. Se o nome estiver armazenado, o programa deve exibir a sua posição. Caso contrário, deve apresentar uma mensagem informando que o nome não foi localizado. Atenção: antes de realizar a verificação, os nomes dentro do vetor devem estar ordenados!
64. Escreva um programa que utilize o método de busca binária para encontrar e imprimir a posição de determinado elemento dentro de um vetor, conforme o exemplo abaixo. Caso o elemento não seja encontrado, o programa deve imprimir o valor -1. Quando a quantidade informada para o número de elementos do vetor for zero, o programa deverá ser encerrado.

```
1 Digite a quantidade de elementos do vetor: 5
2 Digite os elementos do vetor: 1 2 3 4 5
3 Digite o elemento procurado: 3
4 Posição do elemento procurado: 2
5
6 Digite a quantidade de elementos do vetor: 5
7 Digite os elementos do vetor: 4 8 7 2 1
```

```
8 Digite o elemento procurado: 3
9 Posição do elemento procurado: -1
10
11 Digite a quantidade de elementos do vetor: 0
12
13 => Fim do programa!
```

65. Escreva um programa que realize a *pesquisa sequencial* em um vetor de números inteiros. O programa deve solicitar a entrada de números inteiros, até que o valor -999 seja digitado. Em seguida, o programa deve pedir para o usuário digitar o número que deve ser procurado e efetuar a busca sequencial. Se o número foi encontrado, apresentar uma mensagem informando a sua posição dentro do vetor. Caso contrário, apresentar uma mensagem informando que o número não foi encontrado.

66. Escreva um programa que realize a *pesquisa binária* em um vetor de números reais. O programa deve solicitar a entrada de números reais, até que o valor -999 seja digitado. Em seguida, o programa deve pedir para o usuário digitar o número que deve ser procurado e efetuar a busca binária. Se o número foi encontrado, apresentar uma mensagem informando a sua posição dentro do vetor. Caso contrário, apresentar uma mensagem informando que o número não foi encontrado.

Complexidade de Algoritmos

67. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int i, j, k;
2
3  for (i = 0; i < N; i++) {
4      for (j = 0; j < N; j++) {
5          R[i][j] = 0;
6          for (k = 0; k < N; k++) {
7              R[i][j] += A[i][k] * B[k][j];
8          }
9      }
10 }
```

68. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int i, j, k, s;
2
3  for (i = 0; i < N - 1; i++)
4      for (j = i + 1; j < N; j++)
5          for (k = 1; k < j; k++)
6              s = 1;
```

69. Escreva um programa que receba os valores de um vetor de números inteiros e imprima a mensagem “ORDENADO”, caso o vetor estiver em ordem crescente. Qual é a função de custo do pior caso e qual a sua ordem de complexidade O ?

70. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  for (i = 0; i < N; i++) {
2      cout << i << endl;
3  }
```

71. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1 for (i = 0; i < N; i += 2) {  
2     cout << i << endl;  
3 }
```

72. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1 for (i = 0; i <= N; i = 2 * i) {  
2     cout << i << endl;  
3 }
```

73. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1 for (i = 0; i < N; i = i + 2) {  
2     for (j = N - 1; j >= 0; j--) {  
3         if (V[i] < V[j]) {  
4             cout << i << endl;  
5         }  
6     }  
7 }
```

74. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1 int i, j, s;  
2 s = 0;  
3  
4 for (i = 0; i < N - 1; i++)  
5     for (j = 1; j < 2 * N; j++)  
6         s = s + 1;
```

75. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {
2
3      int i, j, m;
4
5      int i = 0;
6      int m = 0;
7
8      while (i < n) {
9          j = 0;
10         while (j < n) {
11             m++;
12             j++;
13             i++;
14         }
15     }
16
17     return m;
18 }
```

76. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  m = 0;
2  i = 0;
3
4  while (i < n) {
5      m++;
6      i++;
7  }
```

77. Escreva um programa que receba valores ordenados e armazene-os dentro de um vetor de números inteiros. Em seguida, o programa deve receber um número e verificar se ele se encontra armazenado dentro desse vetor. Qual é a função de custo do pior caso e qual a sua ordem de

complexidade O ?

78. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {  
2  
3      int i, j, m;  
4  
5      int i = 0;  
6      int m = 0;  
7  
8      while (i < n) {  
9          j = 0;  
10         while (j < i) {  
11             m++;  
12             j++;  
13         }  
14         i++;  
15     }  
16  
17     return m;  
18 }
```

79. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {  
2  
3      int i = 1;  
4      int m = 0;  
5  
6      while (i < n) {  
7          m++;  
8          i *= 2;  
9      }
```

```
10
11     return m;
12 }
```

80. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {
2
3      int i, m;
4
5      i = n;
6      m = 0;
7
8      while (i > 0) {
9          m++;
10         i = i / 2;
11     }
12
13     return m;
14 }
```

81. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {
2
3      int i, j, k, m;
4
5      i = 0;
6      m = 0;
7
8      while (i < n) {
9          j = 0;
10         while (j < n) {
```

```
11         k = 0;
12         while (k < n) {
13             m++;
14             k++;
15         }
16         j++;
17     }
18     i++;
19 }
20
21 return m;
22 }
```

82. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1 int teste(int n) {
2
3     int i, j, k, m;
4
5     i = 0;
6     m = 0;
7
8     while (i < n) {
9         j = 0;
10        while (j < n) {
11            m++;
12            j++;
13        }
14        i++;
15    }
16
17    i = 0;
18
19    while (i < n) {
```

```
20     k = 0;
21     while (k < n) {
22         m++;
23         k++;
24     }
25     i++;
26 }
27
28 return m;
29 }
```

83. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {
2
3     int i, j, m;
4
5     i = 0;
6     m = 0;
7
8     while (i < n) {
9         j = 0;
10        while (j < sqrt(n)) {
11            m++;
12            j++;
13        }
14        i++;
15    }
16
17    return m;
18 }
```

84. Escreva um programa que receba valores ordenados e armazene-os dentro de um vetor de números inteiros. Em seguida, o programa deve receber um número extra e armazená-lo em

sua posição correta, dentro desse vetor. Qual é a função de custo do pior caso e qual a sua ordem de complexidade O ?

85. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {
2
3      int i, j, m;
4
5      i = n;
6      m = 0;
7
8      while (i > 0) {
9          j = 0;
10         while (j < i) {
11             m++;
12             j++;
13         }
14         i /= 2;
15     }
16
17     return m;
18 }
```

86. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {
2
3      int i, j, m;
4
5      i = 0;
6      m = 0;
7
8      while (i < n) {
```

```
9      j = i;
10     while (j > 0) {
11         m++;
12         j--;
13     }
14     i++;
15 }
16
17 return m;
18 }
```

87. Utilizando a notação *Big-O* e considerando o pior caso, calcule a complexidade e encontre o tempo de execução do fragmento de código listado abaixo:

```
1  int teste(int n) {
2
3      int i, j, k, m;
4
5      i = 0;
6      m = 0;
7
8      while (i < n) {
9          j = i;
10         while (j < n) {
11             k = j + 1;
12             while (k < n) {
13                 m++;
14                 k++;
15             }
16             j++;
17         }
18         i++;
19     }
20
21     return m;
}
```

22 }

88. O teorema mestre é utilizado para resolver recorrências na forma $T(n) = aT\left(\frac{n}{b}\right) + f(n)$. Sabendo que o algoritmo *Merge Sort*, também conhecido por ordenação por mistura, tem a forma $T(n) = 2T\left(\frac{n}{2}\right) + n$, encontre a complexidade desse algoritmo.

89. O teorema mestre é utilizado para resolver recorrências na forma $T(n) = aT\left(\frac{n}{b}\right) + f(n)$. Sabendo que o algoritmo de *Busca Binária* tem a forma $T(n) = T\left(\frac{n}{2}\right) + O(1)$, encontre a complexidade desse algoritmo.

90. O teorema mestre é utilizado para resolver recorrências na forma $T(n) = aT\left(\frac{n}{b}\right) + f(n)$. Supondo que um algoritmo tem a forma $T(n) = 4T\left(\frac{n}{2}\right) + n^2$, encontre a complexidade desse algoritmo.

91. Aplique o teorema mestre modificado para encontrar a complexidade de um algoritmo que tem a forma $T(n) = 16T\left(\frac{n}{4}\right) + n$.

92. Aplique o teorema mestre modificado para encontrar a complexidade de um algoritmo que tem a forma $T(n) = 3T\left(\frac{n}{2}\right) + n^2$.

Referências

- [1] **C++ Language**. Disponível em: <<https://cplusplus.com/doc/tutorial>>. Acesso em: 19 de agosto de 2024.
- [2] **C++ Reference**. Disponível em: <<https://en.cppreference.com>>. Acesso em: 19 de agosto de 2024.
- [3] Harvey M. Deitel and Paul J. Deitel. **C++: Como Programar**, 2006. 5 ed. São Paulo: Pearson Prentice Hall, 2006.
- [4] **Learn CPP**. Disponível em: <<https://www.learncpp.com>>. Acesso em: 19 de agosto de 2024.
- [5] José Augusto Navarro Garcia Manzano. **Programação de Computadores com C++: Guia Prático de Orientação e Desenvolvimento**, 2010. 1 ed. São Paulo: Érica, 2010.
- [6] Paulo Giovanni de Faria Zeferino. **Slides da Aula 02 de Programação Orientada a Objetos**, 2024. Material de aula disponibilizado aos alunos que cursaram a disciplina de Programação Orientada a Objetos [PDF].
- [7] Paulo Giovanni de Faria Zeferino. **Slides da Aula 03 de Programação Orientada a Objetos**, 2024. Material de aula disponibilizado aos alunos que cursaram a disciplina de Programação Orientada a Objetos [PDF].